

自适应根树： 主存数据库的巧妙索引

维克多·莱斯，阿方斯·肯珀，托马斯·诺伊曼

福库尔特，信息科技

慕尼黑工业大学

boltzmanstre 3, D-85748 Garching

< lastname > @ in. 鼓所发出之声。德

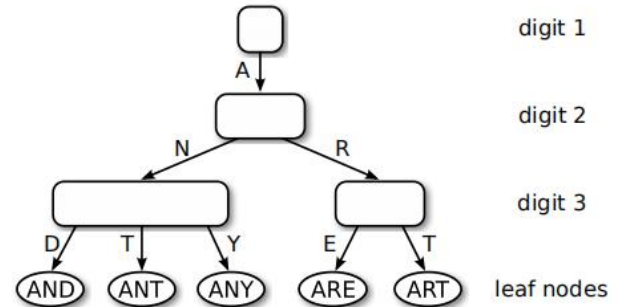
主存容量已经增长到一个点，大多数数据库适合RAM。对于主存数据库系统，索引结构性能是一个关键的瓶颈。传统的内存数据结构(如平衡二叉搜索树)在现代硬件上效率不高，因为它们不能最优地利用cpu上的缓存。哈希表也经常用于主存索引，虽然速度很快，但只支持点查询。

为了克服这些缺点，我们提出了ART，一种在主存中有效索引的自适应基树(trie)。它的查找性能超过了高度调优的只读搜索树，同时也支持非常高效的插入和删除。同时，ART具有很高的空间效率，通过自适应地为内部节点选择紧凑高效的数据结构，解决了困扰大多数根树的最坏情况空间消耗过大的问题。尽管ART的性能与哈希表相当，但它按排序顺序维护数据，从而支持范围扫描和前缀查找等其他操作。

我。介绍

经过几十年主内存容量的增长，即使是大型事务性数据库也适合RAM。当大多数数据高速缓存的传统数据库系统是受CPU限制的，因为它们花费了相当多的精力来避免磁盘访问。这导致了对H-Store/voldb[1]、SAP HANA[2]和HyPer[3]等主存数据库系统的非常激烈的研究和商业活动。这些系统针对新的硬件环境进行了优化，因此速度要快得多。例如，我们的系统HyPer将事务编译为机器码，并消除了缓冲区管理、锁定和锁存的开销。对于OLTP工作负载，生成的执行计划通常是索引操作序列。因此，指标效率是决定性的性能因素。

早在25年前，t树[4]就被提出作为一种内存索引结构。不幸的是，处理器架构的巨大变化使得t树和所有传统的二叉搜索树一样，在现代硬件上效率低下。原因是不断增长的CPU缓存大小和主存速度的差异使得统一内存访问时间的基本假设已经过时。B⁺-树的变种像缓存敏感的B⁺-tree[5]有更多的缓存友好的内存访问模式，但需要更昂贵的更新操作。此外，二进制和B的效率⁺-trees受到了现代cpu的另一个特点的影响：因为比较的结果不容易预测，



无花果。1. 基数树中自适应大小的节点。

现代cpu的长管道会停止，这会在每秒钟比较之后(平均)导致额外的延迟。

传统搜索树的这些问题在最近的数据结构研究中得到了解决，这些数据结构专门设计用于现代硬件架构。k-ary搜索树[6]和快速架构敏感树(Fast)[7]使用数据级并行性与单指令多数据(SIMD)指令同时执行多个比较。此外，FAST使用一种数据布局，通过最佳地利用缓存线和转换Lookaside Buffer (TLB)来避免缓存丢失。虽然这些优化提高了搜索性能，但这两种数据结构都不支持增量更新。对于需要连续插入、更新和删除的OLTP数据库系统，一个明显的解决方案是差分文件(delta)机制，但是，这会导致额外的成本。

哈希表是另一种流行的主存数据结构。与访问时间为 $O(\log n)$ 的搜索树相比，哈希表的访问时间为 $O(1)$ ，因此在主存中要快得多。然而，哈希表很少用作数据库索引。原因之一是哈希表随机分散键，因此只支持点查询。另一个问题是，大多数哈希表不能优雅地处理增长，而是需要在溢出时进行昂贵的重组，复杂度为 $O(n)$ 。因此，当前系统面临着只允许点查询的快速哈希表和功能完备但相对较慢的搜索树之间的不幸权衡。

第三类数据结构，即trie、基数树、前缀树和数字搜索树，如图1所示。

这些数据结构直接使用键的数字表示，而不是散列或比较键。其基本思想类似于许多按字母顺序排列的字典中的拇指索引：可以直接使用单词的第一个字符跳转到以该字符开头的单词。在计算机中，这个过程可以对下一个字符重复，直到找到匹配。作为这个过程的结果，所有的操作都有 $O(k)$ 复杂度，其中 k 是键的长度。在数据集非常大的时代，当 n 的增长速度超过 k 时，拥有独立于 n 的时间复杂度是非常有吸引力的。

在这项工作中，我们提出了自适应基数树(ART)，这是一种快速且空间高效的内存索引结构，专门针对现代硬件进行了调整。虽然大多数基树需要通过设置全局有效的扇出参数来权衡树的高度和空间效率，但ART调整每个单独节点的表示，如图1所示。通过局部适应每个内部节点，同时优化全局空间利用率和访问效率。节点使用少量高效且紧凑的数据结构表示，这些数据结构根据子节点的数量动态选择。另外两种技术，路径压缩和惰性扩展，允许ART通过折叠节点有效地索引长键，从而降低树的高度。

基数树的一个有用的特性是键的顺序不像哈希表那样随机；相反，键是按字典顺序按位排序的。我们将展示如何有效地对典型的数据类型进行重新排序，以支持需要对数据进行排序的所有操作。例如，范围扫描、前缀查找、top- k 、最小值和最大值)。

本工作的贡献如下：

- 提出了一种快速、有效的自适应基树算法
 - 用于主存数据库系统的空间高效的通用索引结构。
- 我们证明了每个密钥的空间消耗是有界的
 - 到52字节，即使对于任意长的密钥也是如此。我们通过实验证明，在实践中，空间消耗要低得多，通常低至8。每个密钥1字节。
- 我们将描述常见的内置数据类型
 - 存储在基数树中，同时保持它们的顺序。
- 我们通过实验评估ART和其他状态
 - 艺术主存索引结构，包括最有效的搜索树建议。
- 通过将ART集成到主存数据库中
 - 系统HyPer和运行TPC-C基准测试，我们在“现实生活”的事务处理应用程序中证明了其优越的端到端性能。

本文的其余部分组织如下。下一节将讨论相关工作。第三节介绍了自适应基数树，并分析了其空间消耗。在第四节中，我们将介绍二进制可比键的概念，并展示如何转换常见的内置类型。第五节描述了实验结果，包括一些微基准测试和TPC-C基准测试。最后，第六节总结并讨论了今后的工作。

在基于磁盘的数据库系统中， B^+ -tree [8]是一种无处不在的[9]。它从磁盘中检索大块以减少访问次数。红黑树[10]、[11]和 t 树[4]被提出用于主存数据库系统。Rao和Ross[5]表明，像所有二叉搜索树一样， t 树的缓存行为很差，因此通常比 B^+ -现代硬件上的树。作为替代方案，他们提出了一个具有缓存意识的 B^+ -tree的变体，CSB $^+$ -tree[12]。对 B 的进一步缓存优化 $^+$ -树木由Graefe和Larson调查[13]。

现代cpu允许使用单个SIMD指令执行多次比较。施莱格尔等人。[6]提出了[6]搜索，它减少了从日志中进行比较的次数 $2N$ 到 $\log_K n$ ，其中 K 是适合一个SIMD向量的键数。与二叉树相比，这种技术还减少了缓存丢失的次数，因为对从主存加载的每条缓存行执行 K 次比较。金等人。提出了FAST，一种以架构敏感的方式布局二叉搜索树的方法[7]。SIMD、缓存行和页面阻塞用于最佳地使用可用的缓存和内存带宽。此外，他们还提出将多个查询的各个阶段穿插在一起，以提高搜索算法的吞吐量。FAST树和 k -ary搜索树是无指针的数据结构，它们将所有键存储在单个数组中，并使用偏移计算来遍历树。虽然这种表示很有效并节省了空间，但它也意味着不可能进行在线更新。Kim等人。还介绍了一种快速的GPU实现，并将其性能与现代cpu进行了比较。他们的研究表明，由于更高的内存带宽，gpu可以实现比cpu更高的吞吐量。然而，使用gpu作为专用索引硬件还不现实，因为gpu的内存容量有限，与主存的通信成本很高，并且需要数百个并行查询来实现高吞吐量。因此，我们专注于cpu的索引结构。

使用尝试对字符串进行索引已经得到了广泛的研究。最早的两个变体使用列表[14]和数组[15]作为内部节点表示。莫里森引入了路径压缩，以便有效地存储长字符串。Knuth [17]分析了这些早期的三种变异。burst tree是最近的一个提议，它在树的上层使用tree节点，但一旦子树只有很少的元素就切换到链表。HAT-trie[18]通过用散列表替换链表来提高性能。虽然大多数研究都集中在索引字符串上，但我们的目标是也索引其他数据类型。因此，我们更喜欢术语基数树而不是trie，因为它强调了与基数排序算法的相似性，并强调可以对任意数据进行索引，而不仅仅是对字符串进行索引。

广义前缀树由Boehm等人提出。[19]作为通用索引结构。它是一个基数
树粉丝16岁，进入SIGMOD决赛

编程比赛2009。KISS-Tree[20]是一个更有效的基数树提案，只有三个级别，但只能存储32位密钥。它对密钥的前16位使用了一个开放的寻址方案，并依赖于虚拟内存系统来节省空间。第二级，负责接下来的10位，使用一个数组表示，最后一级使用一个位向量压缩6位。动态改变内部节点表示的想法是由惠普研究实验室[21]，[22]开发的Judy阵列数据结构所使用的。

Graefe讨论了二进制可比性（“标准化”）键，e。g。[23]，作为一种简化和加速关键比较的方法。我们使用这个概念来获得存储在基数树中的键的有意义的顺序。

III. 自适应半径树

本节介绍自适应基树（ART）。我们首先从一些关于半径树相对于基于比较的树的优点的一般观察。接下来，我们通过展示传统半径树的空间消耗可能过大来激励自适应节点的使用。我们继续描述ART和搜索和插入的算法。最后，对空间消耗进行了分析。

A. 前期准备工作

基数树有许多有趣的属性，使它们区别于基于比较的搜索树：

- 树的高度(和复杂度)取决于键的长度，但一般不在树中元素的数量上。
- 根树不需要再平衡操作
插入顺序会导致相同的树。
- 这些密钥按字典顺序存储。
- 到叶节点的路径表示该节点的键
叶子。因此，键是隐式存储的，并且可以从路径中进行重构。

根树由两种类型的节点组成：内部节点，它将部分键映射到其他节点；和叶节点，它存储与键对应的值。内部节点最有效的表示是作为一个2的数组^s指针在遍历树期间，键的一个s位块被用作该数组的索引，从而确定下一个子节点，而不需要任何额外的比较。参数s（我们称之为span）对于基数树的性能至关重要，因为它决定了给定密钥长度的树的高度：

存储k位密钥的基数树有 $\lceil k/s \rceil$ 级的内部节点。例如，对于32位键，使用 $s = 1$ 的基数树有32个级别，而8个级别的跨度的结果只有4个级别。由于基于比较的搜索树是数据库系统中普遍存在的索引结构，因此在完全平衡的搜索树中，比较半径树的高度与比较的次数是具有说明性的。虽然在最佳情况下，每次比较都排除了所有值的一半，但如果是 >1 ，则一个基数树节点可以排除更多的值。因此，对于 $n > 2$ ，半径树的高度小于二值搜索树 k / s 。图2说明了这种关系，并假设键

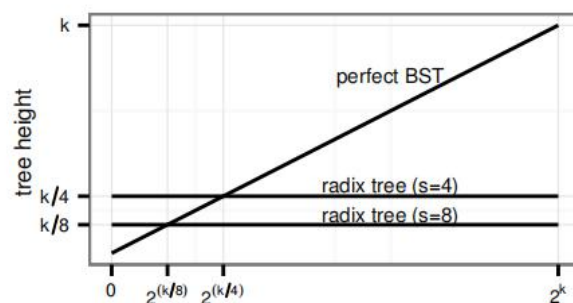
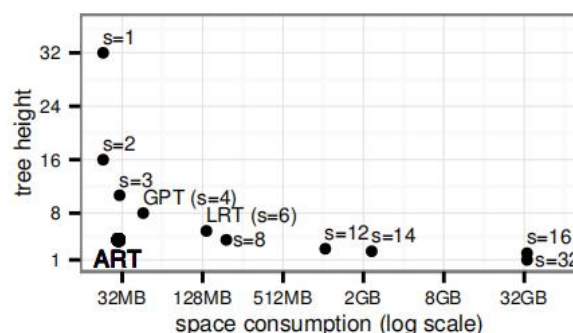


图2所示。完全平衡的二叉搜索树和半径树的树高。

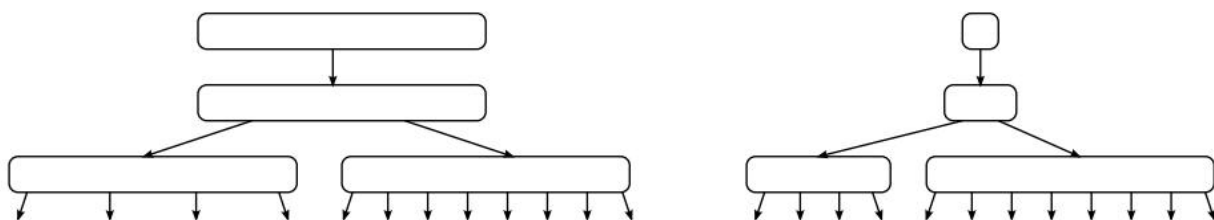
可以在 $O(1)$ 个时间内进行比较。对于大键，比较实际上需要 $O(k)$ 时间，因此搜索树的复杂度为 $O(k \log n)$ ，而不是基数树的复杂度为 $O(k)$ 。这些观察结果表明，半径树，特别是具有大跨度的树，可以比传统的搜索树更有效。

B. 自适应节点

正如我们所看到的，从（纯查找）性能的角度来看，我们希望有一个较大的跨度。当使用指针数组来表示内部节点时，大跨度的缺点也很明显：当大多数子指针为空时，空间的使用可能会过多。图3显示了在存储1M均匀分布的32位整数时，span参数的不同值的树高和空间消耗。随着跨度的增加，树高呈线性减小，而空间消耗呈指数增长。因此，在实践中，只有一些s的值提供时间和空间之间合理的权衡。例如，广义前缀树（GPT）使用4位[19]的跨度，而在Linux内核（LRT）中使用的radix树使用6位[24]。图3进一步显示了我们的自适应基数树（ART），同时，比只使用齐次阵列节点的基数树使用更少的空间和高度也更小。实现空间效率和时间效率的关键思想是自适应地使用具有相同且相对较大跨度的不同节点大小，但具有不同的扇出。图4说明了这一点



无花果。3. 当存储1M均匀分布的32位整数时，span参数s的不同值的树高和空间消耗。指针长度为8字节，节点会延迟展开。



无花果。4. 使用数组节点（左）和我们的自适应基数树ART（右）。

思想和结果表明，自适应节点不影响树的结构（即高度），只影响节点的大小。通过减少空间消耗，自适应节点允许使用更大的跨度，因此也提高了性能。

为了有效地支持增量更新，在每次更新后调整节点大小的成本太大。因此，我们使用少量节点类型，每种类型都有不同的扇出。根据非空子节点的数量，将使用适当的节点类型。当一个节点的容量因插入而耗尽时，它将被一个更大的节点类型替换。相应地，当一个节点由于删除键而变得欠满时，它会被一个较小的节点类型所取代。

C. 内部节点结构

从概念上讲，内部节点将部分键映射到子指针。在内部，我们使用了四种具有不同容量的数据结构。给定下一个键字节，每个数据结构都允许有效地查找、添加和删除子节点。此外，子指针可以按排序顺序扫描，这允许实现范围扫描。我们使用一个8位的跨度，对应于1字节的部分键，并产生一个相对较大的扇出。这种选择还具有简化实现的优点，因为字节可以直接寻址，从而避免了位移位和屏蔽操作。

这四种节点类型如图5所示，并根据它们的最大容量进行命名。我们没有使用键/值对的列表，而是将该列表分割为一个关键部分和一个指针部分。这允许在允许高效搜索的同时保持表示紧凑：

Node4: 最小的节点类型最多可以存储4个子指针，并使用长度为4的数组存储键，使用相同长度的另一个数组存储指针。键和指针存储在相应的位置，并对键进行排序。

节点16: 此节点类型用于存储5到16个子指针。与节点4一样，键和指针也存储在相应位置的单独数组中，但两个数组都有容纳16个条目的空间。通过二进制搜索，或者在现代硬件上，使用SIMD指令进行并行比较，可以有效地找到密钥。

Node48: 随着节点中条目数量的增加，搜索键数组变得昂贵。因此，具有超过16个指针的节点不会显式地存储键。相反，使用一个包含256个元素的数组，可以直接使用键字节对其进行索引。如果一个节点有17到48个子指针，那么这个数组将索引存储到第二个数组中，该数组包含多达48个指针。这种间接方式节省了空间

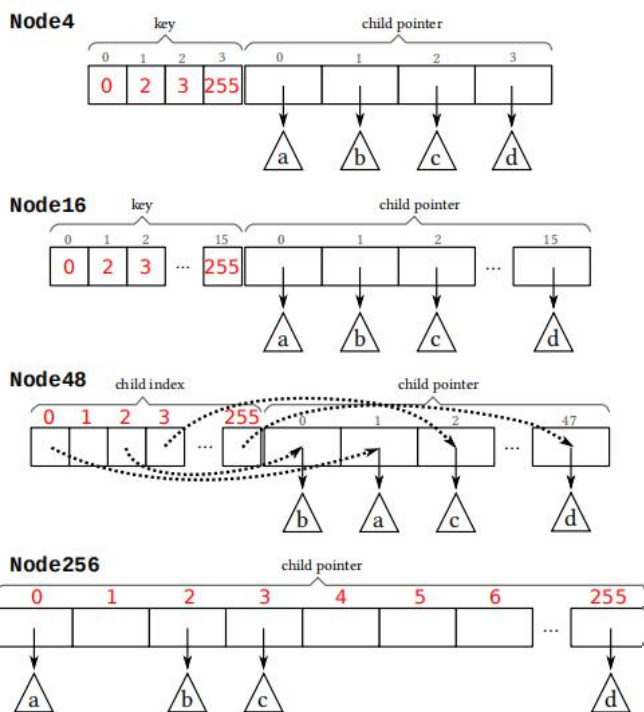


图5. 内部节点的数据结构。在每种情况下，部分键0、2、3和255分别被映射到子树a、b、c和d。

与256个8个字节的指针相比，因为索引只需要6个位（为了简单起见，我们使用1个字节）。

节点256: 最大的节点类型仅仅是一个包含256个指针的数组，用于存储49到256个条目。使用这种表示，可以使用该数组中的关键字节的单个查找来非常有效地找到下一个节点。不需要额外的间接说明。如果大多数条目都不是空的，那么这个表示也非常节省空间，因为只需要存储指针。

此外，在每个内部节点的前面，一个相同大小大小（例如16字节）的头存储节点类型、子节点的数量和压缩路径（cf。第III-E节）。

D. 叶节点结构

除了使用上一节中讨论的内部节点存储路径外，radix树还必须存储与键关联的值。我们假设只存储唯一的键，因为非唯一的索引可以通过将元组标识符附加到每个键上作为连接断路器来实现。

这些值可以以不同的方式进行存储：

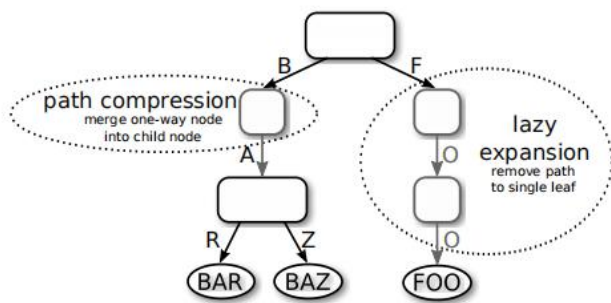


图6. 延迟扩展和路径压缩的说明。

- 单值叶：这些值使用addi-存储存储一个值的任意叶节点类型。
- 多值叶：值存储在四个中的一个不同的叶节点类型，它们反映了内部节点的结构，但包含值而不是指针。
- 组合的指针/值插槽：如果值适合于点-嗯，不需要单独的节点类型。相反，内部节点中的每个指针存储位置都可以存储一个指针或一个值。值和指针可以使用每个指针额外的一位或使用指针标记来区分。

使用单值叶是最一般的方法，因为它允许在一个树中使用不同长度的键和值。但是，由于增加了树的高度，每次查找都会导致一次额外的指针遍历。多值叶避免了这种开销，但要求树中的所有键都具有相同的长度。组合的指针/值插槽是有效的，并允许存储不同长度的键。因此，如果适用，应使用这种方法。它对于存储与指针大小相同的元组标识符的二级数据库索引特别有吸引力。

E. 折叠内部节点

基数树具有一个有用的属性，即每个内部节点表示一个键前缀。因此，键被隐式地存储在树的结构中，并且可以从路径重构到叶节点。这节省了空间，因为不需要显式地存储键。然而，即使使用这种隐式的键前缀压缩和自适应节点，在某些情况下，特别是对于长键，每个键的空间消耗还是很大的。因此，我们使用了另外两种众所周知的技术，它们可以通过减少节点的数量来降低高度。这些技术对于长键非常有效，从而显著提高了这些索引的性能。同样重要的是，它们减少了空间消耗，并确保了一个较小的最坏情况下的空间界。

使用第一种技术，即延迟扩展，只有在需要区分至少两个叶节点时，才能创建内部节点。图6显示了一个示例，其中惰性展开通过截断到叶子“FOO”的路径来节省两个内部节点。如果插入了另一个前缀为“F”的叶，则会展开此路径。注意，因为到叶节点的路径可能被截断，

搜索（节点、键、深度）

- 1如果节点=NULL
- 2返回空
- 3如果isLeaf（节点）
- 4 if leafMatches(节点，键，深度)
- 5返回节点
- 6返回NULL
- 如果checkPrefix(节点，键，深度)!=节点.prefixLen
- 8返回NULL
- 9 depth=depth+node .前缀Len
- 10下一个=find子点（节点，键[深度]）
- 11返回搜索（下一个，键，深度+1）

图7. 搜索算法。

此优化要求键存储在叶子中，或者可以从数据库中检索。

路径压缩是第二种技术，它将删除只有一个子节点的所有内部节点。在图6中，删除了存储部分键“A”的内部节点。当然，不能简单地忽略这个部分键。有两种方法可以处理它：

- 悲观：在每个内部节点，一个可变的长度存储了部分键向量。它包含所有已删除的所有单向节点的键。在查找期间，将此向量与搜索键进行比较，然后再继续查找下一个子向量。
- 乐观性：仅计算前面的单向节点的数量（在悲观方法中等于向量的长度）被存储。查找只需跳过这个字节数，而不进行比较。相反，当查找到达一个叶子时，必须将其键与搜索键进行比较，以确保没有出现“错误的转弯”。

这两种方法都可以确保每个内部节点至少有两个子节点。乐观的方法对长字符串特别有利，但需要一次额外的检查，而悲观的方法使用了更多的空间，并且有可变大小的节点，导致内存碎片增加。因此，我们使用一种混合方法，在每个节点上存储一个向量，就像悲观方法一样，但所有节点的大小不变（8字节）。只有当超过这个大小时，查找算法才会动态地切换到乐观策略。在不浪费太多空间或分割内存的情况下，这避免了在我们调查的情况下的额外检查。

F. 算法

我们现在给出搜索和更新的算法：

搜索：搜索的伪代码如图7所示。通过使用键数组的连续字节遍历树，直到遇到叶节点或空指针。第4行通过检查遇到的叶子是否与键完全匹配来处理延迟扩展。如果压缩路径与键不匹配，则在第7行和第8行中处理悲观路径压缩。下一个子节点由findChild函数找到，如图8所示。根据节点类型执行适当的搜索算法：因为Node4只有2-4个条目，我们使用

```

    findChild (node, byte)
1 如果节点。type==Node4 //简单循环
2 为(i=0; i<节点。计数; 我=i+1)
3 如果节点。key[i]==byte
4 . 返回节点。child[i]
5 返回NULL
6 如果节点。==型节点16//SSE比较
7 key=_mm_setl_epi8(byte)
8 cmp=_mm_cmpeq_epi8(键, 节点。钥匙
9 掩码=(1<<节点。计数) 1
10 bitfield=_mm_movemask_epi8(cmp)&mask
如果位域为11
12 返回节点。child[ctz (bitfield)]
13 其他的
14 返回值为空
15 if 节点。键入==节点48//两个数组查找
16 如果节点。childIndex[byte]!=EMPTY
17 返回节点。child[node .childIndex[byte]]
18 其他
19 返回值为空
20 如果节点。type==Node256 //一个数组查找
21 返回节点。child[byte]

```

图8. 在给定部分密钥的内部节点中查找子节点的算法。

一个简单的循环。对于节点16，伪代码显示了一个使用SSE指令的SIMD实现，它允许将16个键与一个指令并行进行比较。首先，复制搜索的键(第7行)，然后与存储在内部节点中的16个键进行比较(第8行)。在下一步中，将创建一个掩码(第9行)，因为该节点的有效项可能少于16个。比较的结果被转换为位域并应用掩码(第10行)。最后，使用计数尾零指令将位字段转换为索引(第12行)。另外，如果SIMD指令不可用，也可以使用二进制搜索。在节点48中查找时，首先检查子索引项是否有效，然后返回相应的指针。节点256查找只包含单个数组访问。

插入:伪代码如图9所示。使用第29行中的递归调用遍历该树，直到找到新叶的位置为止。通常，叶子可以简单地插入到现有的内部节点中(第31-33行)。如果由于延迟扩展而遇到现有叶，则它被存储现有叶和新叶的新内部节点取代(行513)。如果新叶的键与压缩路径不同，则会出现另一种特殊情况：在当前节点上方创建新的内部节点，并相应地调整压缩路径(第17-24行)。由于空间不足，我们省略了一些辅助函数:replace用另一个节点替换树中的一个节点，addChild向内部节点追加一个新的子节点，checkPrefix将节点的压缩路径与键进行比较并返回相等的字节数，grow用更大的节点类型替换节点，loadKey从数据库中检索叶节点的键。

批量加载:当为现有关系创建索引时，可以使用以下递归算法来加快索引构建:使用每个键的第一个字节，将键/值对基数划分为256个分区和1个分区

```

    插入(节点、键、叶子、深度)
1, 如果节点==NULL//处理空树
2 . 替换(节点、叶片)
3 返回
4, 如果是Leaf (节点) //展开节点
5         newNode = makeNode4 ()
6 个键2个=加载键 (节点)
7 为 (i=深度; 键[i]==键2[i]; i=i+1)
8 newNode .prefix[i-depth]=key[i]
9 newNode .prefixLen = i-depth
10 . depth=depth+newNode. 前缀Len
11 加法子节点 (新节点、键深度、叶)
12 附加子 (新节点, 键2[深度, 节点)
13. 更换 (节点、新节点)
14 返回
15 p=checkPrefix(节点, 键, 深度)
16 如果p!=节点。前缀不匹配
17 newNode=makeNode4 ()
addChild(newNode, key[depth+p], leaf)
19 添加的孩子(新的节点, 节点。前缀[p], 节点)
20 newNode. prefixLen=p
21 memcpy (newNode .前缀, 节点。前缀, p)
22 节点。prefixLen=node .prefixLen——(p + 1)
23 memmove(节点。前缀, 节点。前缀+ p + 1, 节点。prefixLen)
24 替换 (节点、新节点)
25 返回
26 depth=depth+node .prefixLen
27 下一个=find子点 (节点, 键[深度])
28 if next //递归
插入(next, key, leaf, depth+1)
30 else //添加到内部节点
31, 如果为Full (节点)
32         成长(节点)
33 加法子节点 (节点、键深度、叶)

```

无花果。9. 插入算法。

创建适当类型的内部节点。在返回该内部节点之前，将通过使用每个键的下一个字节递归地为每个分区应用批量加载过程来创建它的子节点。

删除:删除操作的实现与插入操作对称。从内部节点移除叶子，必要时将其缩小。如果该节点现在只有一个子节点，则它将被其子节点替换，并调整压缩路径。

G. 空间消耗

尽管具有tb内存的服务器很容易获得，但主存仍然是一种宝贵的资源。因此，索引结构应该尽可能紧凑。对于基数树，空间消耗取决于存储的密钥的分布。密钥(e.g., 从1到n)的整数是最好的情况，并且可以有效地存储空间，即使使用大跨度和不使用自适应节点。另一方面，当键是稀疏的时候，内部节点的许多指针为空，这会导致空间浪费。倾斜的键会导致一些节点主要包含空指针，而其他节点则会密集排列。自适应节点解决了这些问题，并通过局部优化每个节点的空间消耗，确保任何密钥分布都被紧凑地存储。

现在我们分析每个键的最坏情况空间消耗，考虑到自适应节点、延迟扩展和路径压缩。对于下面的分析，我们假设指针有8字节长，每个节点有16字节长

表i
节点类型的摘要(16字节标头, 64位指针)。

类型	儿童	空间 (字节)
Node4	2 - 4	16 + 4 + 4 8 = 52.
Node16	516	16 + 16 + 16 8 =
Node48	17-48	160 16 + 256 + 48 8
Node256	49-256	= 656 16 + 256 8 = 2064

表二
最坏情况下, 每个键的空间消耗 (字节) 的不同
具有64位指针的RADIX树变体。

	k = 32	k ! 1
艺术	43	52
GPT	256	1
LRT	2048	1
技能	>4096	NA.

存储节点类型、非空子节点的数量和压缩路径的标头。我们只考虑内部节点而忽略叶节点, 因为如果使用带有标记指针的组合指针/值槽, 叶就不会产生空间开销。使用这些假设, 每个内部节点类型的结果空间消耗如表I所示。

将每个叶视为提供x字节, 将内部节点视为子节点提供的消耗空间。如果树的每个节点都有一个正预算, 那么该树每个键使用的字节少于x个字节。内部节点的预算是其子节点的所有预算之和减去该节点的空间消耗。形式定义有子节点c(n)和空间消耗s(n)的节点n的预算b(n)为

$$b(n) = \begin{cases} x, & \text{isLeaf}(n) \\ \left(\sum_{i \in c(n)} b(i) \right) - s(n), & \text{else.} \end{cases}$$

使用节点类型的归纳, 我们现在显示b (n)对于每个ART节点n的52, 如果x = 52: 对于叶子, 根据预算函数的定义。 $\geq$ 为了证明该语句适用于内部节点, 我们计算了对的一个下界 $i2c(n)B(i)$ 使用归纳假设和每个节点类型的最小子节点数。在减去相应的空间消耗后, 我们得到了每个节点类型的预算的一个下界。在所有四种情况下, 它都大于或等于52, 即得出归纳法。总之, 我们已经证明, 对于x = 52, 不可能构造一个具有负预算的自适应基数树节点。因此, 对于任何自适应基数树, 最坏情况下的空间消耗是52字节, 即使对于任意长的键也是如此。它也可以类似地显示为有六种节点类型¹, 最坏情况下的空间消耗可以减少到每个键34字节。正如我们将在第V-D节中所展示的, 在实践中, 空间消耗比最坏的情况要小得多, 即使是对于相对较长的键。然而, 8.1字节的最佳情况确实非常频繁, 因为代理整数键通常是密集的。

¹节点4类型被新的节点类型节点2和节点5取代, 节点48类型被新的节点32和节点64类型取代。

让我们最后与表二中总结的其他半径树进行比较。因为广义的前缀树

而Linux内核基数树不使用路径压缩, 内部节点的数量与键的长度成正比。因此, 最坏情况下每个键的空间消耗是没有任何限制的。此外, 即使对于短键, 这两种数据结构都比艺术在最坏情况下的空间消耗要高得多, 因为它们不使用自适应节点。KISS-Tree的最坏情况下的空间消耗是每个键超过4KB, 例如, 使用无符号整数键 $\{i \cdot 2^{16} \mid i \in \{0, 1, \dots, 2^{16}-1\}\}$ 。

增值构造二进制可比键

选择索引结构的一个重要方面是数据是否按排序顺序存储。索引结构的排序顺序遍历有助于实现高效的排序范围扫描和最小、最大、顶部n等值的查找。默认情况下, 只有基于比较的树按排序的顺序存储数据, 这导致它们普遍作为数据库系统的索引结构。虽然已经提出使用保持顺序的哈希来允许对哈希表元素进行排序, 但这在现实世界的系统中并不常见。原因是, 对于来自未知分布的值, 很难想出能够在保持输入顺序的同时将输入值均匀地分布在哈希表上的函数。

存储在基数树中的密钥是按位字典顺序排列的。对于某些数据类型, e. g., 通过对ASCII进行编码的字符串, 这就产生了预期的顺序。对于大多数数据类型, 情况并非如此。例如, 负二补有符号整数在字典学上大于正整数。但是, 可以通过转换键来获得所需的顺序。我们称这种转换二进制可比键产生的值。如果只将二进制可比键用作基数树的键, 则数据将按排序顺序存储, 并支持依赖于此顺序的所有操作。请注意, 对上一节中介绍的算法无需进行任何修改。在存储或查找每个键之前, 必须简单地将其转换为二进制可比键。

二进制可比键还有其他用例。就像这个概念允许用基数树替换基于比较的树一样, 它也允许用基于比较的排序算法, 如快速排序算法或合并排序算法替换为基数排序算法。

一个. 定义

一个转换函数 $t: D \rightarrow \{0, 1, \dots, 255\}^k$ 如果结构域D满足以下等效值 $(x, y \in D)$, 则将其值转换为长度为k的二进制可比键:

- $x < y$, $\text{memcmp}_k(t(x), t(y)) < 0$
- $x > y$, $\text{memcmp}_k(t(x), t(y)) > 0$
- $x = y$, $\text{memcmp}_k(t(x), t(y)) = 0$

运算符 $<$ 、 $>$ 、 $=$ 表示输入类型上通常的关系运算符 $_k$ 比较两个输入向量的分量。如果返回所有比较值, 则返回0

为相等，如果第一个向量的第一个不相等的值小于第二个向量的相应字节，则为负值，否则为正值。

对于有限域，总是可以将任何严格完全有序域的值转换为二进制可比键：一个域大小 n 的每个值都映射到一个 $\lceil \log_2 n \rceil$ 位存储零扩展秩减去1的位置。

B. 转换

现在我们讨论如何将常见的数据类型转换为二进制可比较的键。

a) 无符号整数：无符号整数的二进制表示已经具有所需的顺序。但是，在将值存储到主存时必须考虑机器的特性。在小环境机器上，必须交换字节顺序，以确保结果从最重要字节到最小重要字节排序。

b) 有符号整数：必须重新对有符号的二补体整数进行排序，因为负整数是按降序排列的，并且大于正值。一个 b 位整数 x 通过翻转非常有效地转换符号位（使用 $x \oplus 2^{b-1}$ ），然后将生成的值作为一个无符号值进行存储。

c) IEEE 754浮点数：对于浮点值，转换更复杂，尽管在概念上并不困难。每个值首先被分类为正或负，以及归一化数、非归一化数、NaN、1或0。因为这10个类不重叠，所以可以很容易地计算出一个新的秩，然后像一个无符号值一样存储。一个关键转换需要3个if语句，1个整数乘法和2个加法。

d) 字符串：Unicode排序算法 (UCA) 定义了比较Unicode字符串的复杂规则。有一些开源实现了这个算法，并提供了将Unicode字符串转换为二进制可比较键的函数²。一般来说，每个字符串都以一个不会出现在任何字符串（e.g., 0字节）。原因是键不能是其他键的前缀。

e) Null：要使空值二进制可比，必须给它分配一个具有特定等级的值。对于大多数数据类型，已经使用了所有可能的值。一个简单的解决方案是将所有值的键长度增加1，以获得空值的空间，例如，4字节的整数变成5字节长。适应空值的一种更有效的方法是只增加域的某些值的长度。例如，假设null应该小于所有其他4字节整数，则null可以映射到字节序列0, 0, 0, 0, 0，之前最小的值0映射到0, 0, 0, 0, 1，而所有其他值保留其4字节表示。

f) 复合键：由多个属性组成的键可以通过分别转换每个属性和连接结果来很容易地处理。

²C/C++库“为Unicode“提供的国际组件” (<http://site.icu-project.org/>)，例如，它为此目的提供了ucol_getSortKey函数。

在本节中，我们将实验评估ART，并将其性能与其他内存数据结构进行比较，包括基于比较的树、哈希树和基数树。评估有两个部分：首先，我们执行一些微基准测试，作为独立的程序实现，并使用所有评估的数据结构。在第二部分，我们将一些数据结构集成到主存数据库系统HyPer中。这允许我们执行更实际的工作负载，即标准OLTP基准测试TPC-C。

我们使用的是一个高端桌面系统，它的CPU是Intel酷睿i7 3930K，有6核，12线程，3.2 GHz时钟速率和3.8 GHz涡轮频率。该系统有12 MB的共享、未级缓存和32 GB的四通道DDR31600 RAM。我们使用了64位模式下的Linux 3.2作为操作系统和GCC 4.6作为编译器。

作为参赛者，我们使用

- a B⁺-tree为主存优化（缓存敏感）B⁺-树，
- 两个只读搜索结构优化的现代x86个cpu（k-ary搜索树，快速架构敏感树），
- 基数树（广义前缀树），和。两个教科书数据结构（红黑树[RB]，链接哈希表[HT]使用MurmurHash64A为64位平台[25]）。

为了进行公平的比较，我们使用了作者提供的源代码，如果它可用的话。这就是CSB的情况⁺树[26]，k-ary搜索[27]，和广义前缀树[28]。我们对剩余的数据结构使用了我们自己的实现。

我们能够验证我们提供的FAST实现与最初发布的数字相匹配。为了校准不同的硬件，我们使用了发表在同一论文中的k-ary搜索的结果。我们的FAST实现使用了2 MB的内存页面，并将所有的缓存线块对齐到64个字节的边界，正如Yamamamuro等人所建议的那样。[30]。但是，由于FAST和k-ary搜索返回的是键的秩，而不是元组标识符，因此下面的结果包括在一个单独的元组标识符数组中进行的额外查找以便在数据库上下文中评估有意义的查找。我们必须使用32位整数作为微基准测试的键，因为一些实现只支持32位整数键。对于这种非常短的键，路径压缩通常会增加而不是减少空间消耗。因此，我们为微基准测试删除了这个特性。在更现实的第二部分启用路径压缩。与基于比较的树和哈希表相比，基数树的性能随键的分布而变化。因此，我们显示了从1到 n 的密集键（ n 表示#键中的树的大小）和稀疏键的结果，其中每个位的等可能是0或1。我们随机排列了密集的键。

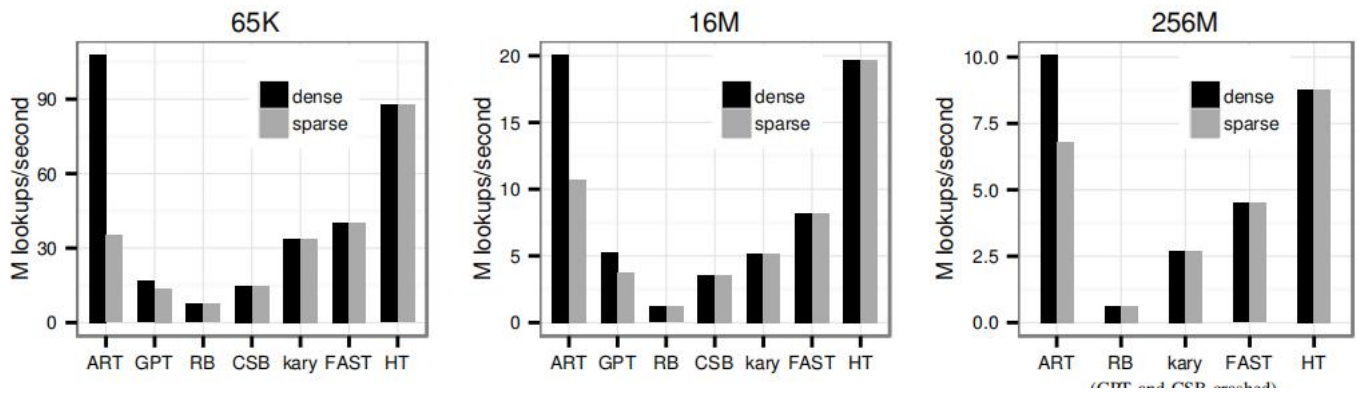


图10. 具有65K、16M和256M键的索引中的单线程查找吞吐量。

一个。搜索性能

在我们的第一个实验中，我们测量了随机查找现有数据的性能³。图10显示了ART和哈希表的最佳性能。ART的速度是GPT的两倍多，GPT是根树，它的跨度是GPT的一半，因此高度是GPT的两倍。红黑树是最慢的基于比较的树，其次是CSB⁺-tree，k-ary搜索，最后是FAST。尽管FAST不支持更新，并且针对现代架构进行了优化，但它比ART和哈希表要慢。对于这三种索引大小的数据结构的相对性能非常相似。这表明，小索引（65K）比大索引（256M）快大约10倍，这主要是由缓存效应造成的，而不是由数据结构的渐近特性造成的。

为了更好地理解这些结果，请考虑表三，它显示了三个最快的数据结构（ART、FAST和哈希表）的每次随机查找的性能计数器。对于16M键，只有部分索引结构被缓存，并且查找是内存绑定的。对于FAST、散列表和带有稀疏键的ART，缓存丢失的数量类似。对于密集的键，ART只会导致一半的缓存丢失，因为它的紧凑节点可以有效地缓存。在小树中，查找性能主要是由指令的数量和分支的错误预测所决定的。虽然ART对于密集键几乎没有错误预测的分支，但稀疏键导致每次查找大约0.85个错误预测的分支，这发生在节点类型调度期间。致密键也需要更少的指令，因为在节点256中查找下一个子节点不需要计算，而在另一个子节点中节点类型会导致更多的努力。FAST有效地避免了普通搜索树中出现的错误预测，但需要大量的指令（每次比较大约5条）来实现这一点。哈希表有一些在碰撞处理过程中发生的少量错误预测。

到目前为止，查找是在一个线程中一次执行一个查询的。下一个实验的目标是找到最大的可实现的吞吐量使用多个

³在基数树中的成功搜索比不成功的搜索要慢。

表三
每次查找的性能计数器。

	65K		16M		
	艺术 (d./s.)	快速的	艺术 (d./s.)	快速的	
周期指令错误	40/10	94	188/35	461	19
。分支L3命中	5	44 75	2	11	1 26
L3未命中	85/12	26 0.0	88/9	0	0 2
	7	0.26	9	0.	2.2
	0.0/0.	4.7	0.0/0.	0	5.1
	85	2.2 0.0	84	2.	.4
	0.65/1	0.0	2.6/3.	5	
	.9		0	2.4	
	0.0/0.		1.2/2.		
	0		6		

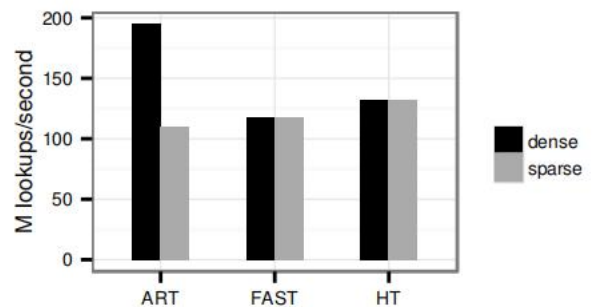


图11. 在一个具有16M个键的索引中的多线程查找吞吐量（12个线程，软件流水线，每个线程有8个查询）。

未同步的线程。除了使用多个线程外，已经证明可以通过使用软件流水线[7]交错多个树遍历来提高吞吐量。该技术更好地利用了现代超尺度cpu，但增加了延迟，并且仅在可用多个查询（e. g.，在使用索引嵌套的循环连接过程中）。FAST从软件流水线（2.5x）中获益最大，因为它相对较大的比较和地址计算延迟可以被有效地隐藏。由于ART首先执行较少的计算，因此它的加速较小，但仍然显著（1.6x1.7x）。链接的哈希表可以被认为是只有两个级别的树（哈希表数组和键/值对的列表），因此加速相对较小（1.2倍）。然而，图11显示，即使每个线程有12个线程和8个交错查询，对于稀疏键，ART只比FAST稍微慢一些，但对于稠密键，ART要快得多。

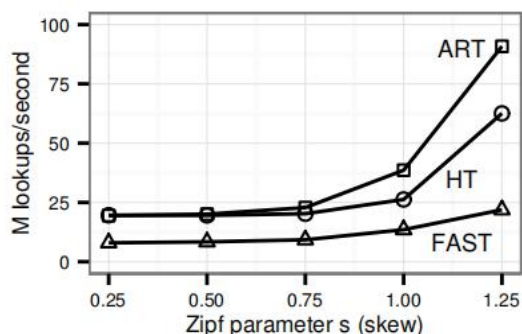


图12. 倾斜对搜索性能的影响 (16M个键)。

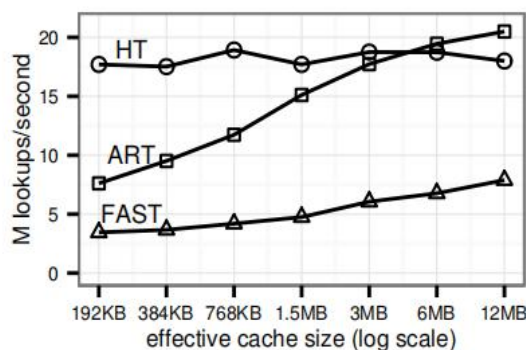


图13. 缓存大小对搜索性能的影响 (1600M密钥)。

B. 缓存效果

现在让我们来研究一下缓存的效果。对于现代CPU来说，缓存是非常重要的，因为DRAM延迟需要数百个CPU周期。特别是树结构从缓存中获益，因为经常访问的节点和顶层通常被缓存。为了量化这些缓存效果，我们将两种树状结构ART（带有密集键）和FAST与哈希表进行了比较。

到目前为止，我们执行的随机查找对于缓存是最糟糕的情况，因为这种访问模式具有糟糕的时间局部性。在实践中，倾斜的访问模式非常常见，例如，最近的订单比旧的订单更容易被访问。我们通过查找Zipf分布式密钥而不是随机密钥来模拟这样的场景。图12显示了倾斜增加对三种数据结构性能的影响。所有数据结构在存在倾斜的情况下都表现得更好，因为缓存丢失的数量会减少。随着倾斜度的增加，ART和哈希表的性能在小的缓存常驻树中接近它们的速度。对于FAST，加速速度较小，因为它需要更多的比较和偏移量计算，而缓存并不能改善这些计算。

现在我们把注意力转向缓存的影响。在之前的实验中，我们只在单个树中执行查找。因此，整个缓存都被利用了，因为没有相互竞争的内存访问。在实践中，缓存通常包含多个索引和其他数据。为了模拟竞争访问，从而有效地减小缓存，我们以循环的方式查找多个数据结构中的键。每个数据结构存储16M随机、密钥，占用超过128MB。图13显示，哈希表基本上不受影响，因为它无论如何都不能有效地使用缓存，而树的性能随着缓存大小的增加而提高，因为更多的时间遍历路径被缓存。使用其中的高速缓存（192KB），ART仅达到整个缓存（12MB）性能的三分之一左右。

C. 更新

除了高效的搜索之外，索引结构还必须支持有效的更新。图14显示了时的吞吐量：

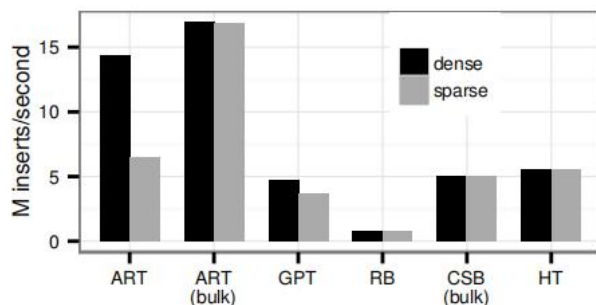


图14. 将16M个键插入到一个空的索引结构中。

在16M的空结构中插入随机键。尽管ART必须在树的增长时动态地替换其内部数据结构，但它比其他数据结构更有效。自适应节点对插入性能的影响（与仅使用节点256相比）的影响为20%。由于自适应节点节省的空间可能很大，因此这通常是值得的权衡。与增量插入相比，批量插入将稀疏键的性能提高了2.5倍，将密集键的性能提高了17%。当插入排序键（例如代理主键）时，由于缓存效果，有序搜索树的性能会提高。对于ART，每秒可以插入5000万个已排序的密集键。只有散列

表不能从排序的顺序中获益，因为哈希随机化了访问模式。

FAST和k-ary搜索树是静态数据结构，只能通过重建它们来更新，这就是为什么它们没有包含在之前的实验中。在需要增量更新的应用程序中使用只读数据结构的一种可能性是使用增量机制：第二种数据结构，它支持在线更新，存储差异，并定期与只读结构合并。为了评估这种方法的可行性，我们使用红黑树存储delta+FAST作为主要搜索结构，并将其与ART（带密集键）和哈希表进行比较。之间的最优合并频率

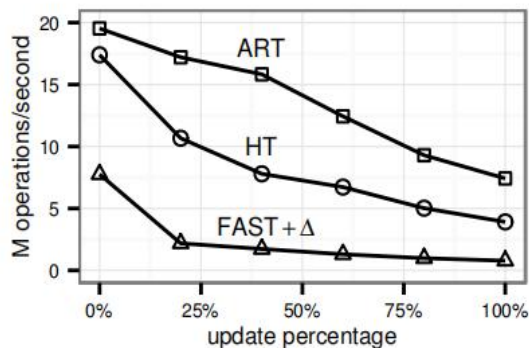
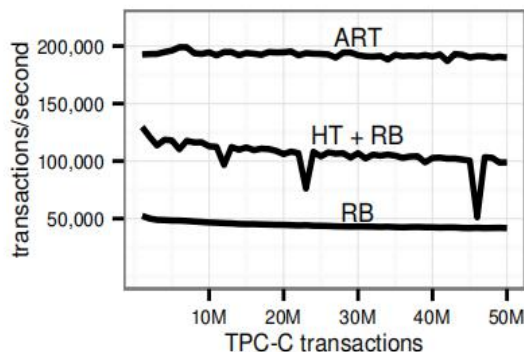


图15. 混合查找、插入和删除(16M键)。



无花果。16. TPC-C性能。

快速和我们通过实验确定的增量。我们的工作负载包括在大约1600万元素的树中的随机插入、删除和查找。图15显示了查找次数与插入和删除次数不同的结果。随着查找次数的减少，由于额外的周期 $O(n)$ 合并步骤，FAST+delta的性能会下降。

D. 端到端评估

在端到端应用程序实验中，我们使用了主存数据库系统HyPer。它的一个独特特点是，它非常有效地同时支持事务（OLTP）和分析（OLAP）工作负载[3]。事务是用脚本语言实现的，该脚本语言被编译成类似汇编程序的LLVM指令[31]。此外，HyPer没有缓冲区管理、锁定或锁的开销。因此，它的性能严重依赖于所使用的索引结构的效率。对于每个索引，HyPer允许确定使用了哪个数据结构。最初，可以使用红黑树和哈希表。我们还集成了ART，包括路径压缩和延迟扩展优化。我们进一步实现了第四节中讨论的所有内置类型的关键转换方案，以便范围扫描、前缀查找、最小操作和最大操作都可以按预期工作。

下面的实验使用了TPC-C，这是一个标准的OLTP基准，模拟了一家管理、销售和分销产品的商品销售公司。它由多种混合的选择语句（包括点查询、范围扫描和前缀查找）、插入和删除语句组成。虽然它被认为是一个重写的基准测试，但54%的SQL语句是查询[32]。我们的实现省略了客户端的思考时间，并使用一个带有5个仓库的分区。我们执行基准测试，直到大部分可用的RAM都是耗尽的作为索引配置，我们使用了ART、红黑树以及哈希表和红黑树的组合。不可能对所有索引使用散列表，因为TPC-C需要对某些索引进行基于前缀的范围扫描。

图16显示了索引结构的选择对HyPer的OLTP性能至关重要。ART的速度几乎是散列表/红-黑树组合的两倍，几乎是四倍

表iv

主要tpc-c索引和使用art的每个键的空间消耗。

#	关系	基数	属性类型	空间
1	条	100,000	int	8.1
2	顾客	150,000	int, int, int	8.3
3	顾客	150,000	int, int, varchar (16), varchar (16), TID	32.6
4	股票	500000 年	int, int	8.1
5	次序	22,177,650	int, int, int	8.1
6	订单	22,177,650	int, int, int, int, TID	24.9
7	指令线	221,712,415	int, int, int, int	16.8

就像红黑相间的树一样快。与红黑树相比，散列表显著提高了性能，但引入了不可接受的重散列延迟，这些延迟在图中作为峰值清晰可见。

让我们将注意力转向我们在测试结束时测量的TPC-C基准测试的空间消耗

基准运行。为了节省空间，HyPer的红黑树和哈希表实现不存储键，只存储元组标识符。使用元组标识符，键根据需要从数据库加载。尽管如此，尽管在最坏的情况下ART可能会为每个元素使用更多的空间，但ART使用的空间仅为哈希表和红黑树的一半。通过考虑每个主要指标的结构信息及其每个键对应的空间消耗情况，可以获得更详细的空间消耗情况，如表四所示。Index 3使用了每个键的大部分空格，因为它存储了相对较长的字符串。然而，它的空间消耗仍然远低于52字节的最坏情况。索引1、2、4和、5仅使用8。每个键1字节，因为它们存储相对密集的整数。指数6和指数7介于这两个极端之间。

我们最后的实验结果，如图17所示，测量了路径压缩和惰性扩展对平均树高的影响。默认情况下，基数树的高度等于键的长度(ART以字节为单位)。例如，如果没有任何优化，索引3的高度将为40。路径压缩和延迟扩展将平均高度降低到8.1。延迟扩展对于长字符串尤其有效。g.，和非唯一的

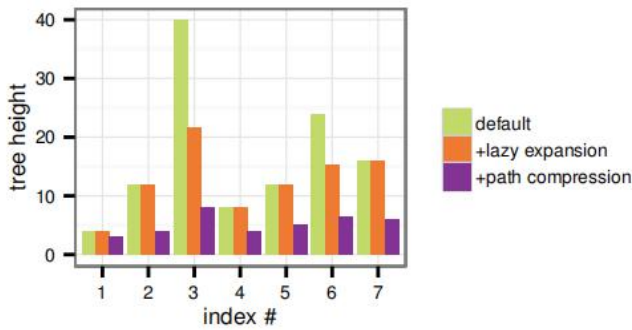


图17. 延迟扩展和路径压缩对TPC-C指标高度的影响。

索引主要包含唯一的值，因为附加的8字节元组标识符可以被截断(e. g., 指数6)。路径压缩有助于处理长字符串(e. g., 以及具有共同前缀的密集整数的复合索引。G ., 索引2、4、5和7)。这两种优化对空间消耗的影响类似于对高度的影响，这就是为什么我们不单独显示它。总之，除了短整数外，路径压缩和延迟扩展对于使用基数树实现高性能和小内存消耗至关重要。

VI . 结论和未来的工作

我们提出了自适应基数树 (ART)，这是一种快速、空间高效的主存数据库系统的索引结构。高扇形、路径压缩和延迟扩展会降低树的高度，从而导致优异的性能。最坏情况下的空间消耗是半径树的一个常见问题，动态选择紧凑的内部数据结构会限制它。我们将ART与其他最先进的主存数据结构进行了比较。我们的结果表明，ART比红黑树(缓存敏感B)快得多⁺树，和GPT，另一个基数树的建议。即使是专门为现代cpu设计的对架构敏感的、只读搜索树的FAST，也比ART慢，即使没有考虑更新。在所有被评估的数据结构中，只有一个哈希表是具有竞争力的。但是哈希表是无序的，因此不适合作为通用的索引结构。通过将ART集成到主内存数据库系统HyPer中并执行TPC-C基准测试，我们证明了它是事务性工作负载的传统索引结构的更好替代方案。

在将来，我们打算着手同步并发更新。特别是，我们计划使用原子原语(如比较-交换)开发无锁存同步方案。另一个想法是设计一个具有相同大小的节点的高效空间半径树。与其根据键的稀疏性动态调整扇形，从键中使用的位数应该动态变化，而扇形则保持近似不变。这样的树也可以用于存储在磁盘上的数据。

[1]
[2]
[3]
[6]
[7]
[8]
[9]
[10]
[11]
[12]
[13]
[14]
[15]
[16]
[17]
[18]
[19]
[20]
[21]

[22]
[23]
[24]
[25]
[26]
[27]
[28]
[29]
[30]
[31]
[32]

参考文献

- R. 卡尔曼, H. 木村, J. Natkins), Pavlo. Rasin S. Zdonik, E. P. C. 琼斯, S. 马登, M. 石材制造商, Y. 张, J. 哈格, 还有 D. J. Abadi, “h-store: 一个高性能的分布式主存事务处理系统”, PVLDB, vol. 1, 2008.
- F. M. Frber, S. K. Cha, J. Primsch, C. Bornhvd, S. Sigg和W. Lehner, “SAP HANA 数据库:现代商业应用的数据管理”, SIGMOD Record, 卷. 40岁, 没有. 4, 2012. 一个. 肯珀和T. 诺伊曼, “炒作: 一个基于虚拟内存快照的混合OLTP和OLAP的主内存数据库系统”, 在ICDE, 2011年.
- T. J. 雷曼兄弟和M. J. 凯里, “对主内存数据库管理系统的索引结构的研究”, 发表在VLDB, 1986年.
- J. 饶和K. A. Ross, “主存中决策支持的缓存有意识索引”, VLDB, 1999年.
- B. 内, R. Gmetla和W. 莱纳, “现代处理器上的k-ary搜索”, 在DaMoN研讨会, 2009年.
- C. 金, J. Chhugani, N. 满意, E. Sedlar, D. 阮, T. Kaldewey, V. W. 李, S. 一个. 勃兰特和P. Dubey, “快速: 在现代cpus和gpus上的快速建筑敏感树搜索”, 发表在SIGMOD, 2010年.
- R. 拜耳和E. McCreight, “大有序索引的组织与维护”, vol. 7, 1997.
- D. Comer, “无处不在的b树”, ACM Comp. 服务., 卷. 11日, 没有. 2, 1979.
- R. “对称二元b树: 数据结构与维护算法”, 信息报, 第1卷. 1, 1972.
- L. 圭巴斯和R. “平衡树的双色框架”, IEEE计算机科学基础年度研讨会, 第1卷. 0, 1978.
- J. Rao和K. 一个. Ross, “使B+树在主存中具有缓存意识”, SIGMOD, 2000.
- G. 格雷夫和P. -A. “b树索引和CPU缓存”, 在ICDE中, 2001.
- R. 论文, “使用可变长度键搜索文件”, 西方联合计算机会议. 1959年.
- E. 弗雷德金, 《特里的记忆》, 《评论》. ACM, 卷. 1960年9月3日.
- D. R. “帕特里夏-检索用字母数字编码的信息的实用算法”, J. ACM, 卷. 15日, 没有. 4, 1968.
- D. E. Knuth, 计算机编程的艺术, 第3卷: (第二版) 排序和搜索, 1998年.
- N. 肩胛骨炎和R. Sinha, “字符串的工程可扩展、缓存和空间效率尝试”, 《VLDB 期刊》, 卷. 19日, 没有. 5, 2010.
- M. Bhm, B. 内, P. B. Volk U. 费舍尔, D. 哈比希和W. Lehner, “使用广义前缀树的高效内存索引”, 顺便说一句, 2011年.
- T. 基辛格, B. 施莱格尔, D. Habich和W. Lehner, “吻树: 现代架构上的智能无锁内存索引”, DaMoN研讨会, 2012年.
- D. 巴士队, “一个10分钟的描述如何工作和为什么他们这么快”, <http://judy.sourceforge.net/doc/10minutes.htm>, 2004.
- A. Silverstein, “Judy IV商店手册”, http://judy.sourceforge.net/judy_interim.pdf, 2002.
- G. Graefe, “在数据库系统中实现排序”, ACM computing. 服务., 卷. 38岁, 没有. 3, 2006.
- J. 树, 树, 树. net/articles/175432/.
- A. Appleby, “MurmurHash64A”, <https://sites.google.com/site/murmurhash/>.
- J. Rao, “CSB+树源”, <http://www.cs.columbia.edu/~kar/software/csb/>.
- B. 施莱格尔, “K-ary搜索消息来源”, <http://wwwdb.inf.tu-dresden.de/team/staff/dipl-inf-benjamin-schlegel/>. M. “广义前缀树源”, <http://wwwdb.inf.tu-dresden.de/research-projects/projects/dexter/>的核心、索引、结构和技术/.
- V. Leis, “FAST来源”, <http://www-db.in.tum.de/~leis/index/fast.cpp>.
- T. Myamamuro, Onizuka, T. Hitaka和M. 山, “VASTTree: 用于大量数据树遍历的向量高级压缩结构”, EDBT, 2012.
- T. 诺伊曼, “高效地编译高效查询计划”, PVLDB, 卷. 4, 2011.
- J. 克鲁格, C. 金, M. 格伦德, N. Satish D. 施瓦尔布, J. Chhugani, H. 普拉特纳先生, P. 杜贝和A. Zeier, “使用多核cpu快速更新读取优化数据库”, PVLDB, vol. 5, 2011.